BRUTE FORCE SSH ATTACK DETECTION & MONITORING

Security Operations Center (SOC) Project Report

Tool	Splunk Enterprise SIEM
Author	Ishtiaq
MITRE ATT&CK	T1110 — Brute Force
Platform	Ubuntu Linux + Splunk Enterprise

1. Executive Summary

This report documents the design, implementation, and results of a Brute Force SSH Attack Detection project using Splunk Enterprise as the Security Information and Event Management (SIEM) platform. The project was carried out in a controlled local lab environment on Ubuntu Linux.

The project involved three core phases:

- Attack Simulation: SSH login attempts were made repeatedly to a local victim account at 127.0.0.1, generating real authentication failure logs in /var/log/auth.log. This was initially done manually, then upgraded to a scripted loop generating over 35 failed attempts within a single minute.
- Log Ingestion and Detection: Splunk Enterprise was configured to ingest the auth.log file and index failed password events. A custom SPL detection query was written, starting with a basic count threshold, then upgraded to time-based detection using per-minute bucketing for more realistic brute-force identification.
- Alerting and Visualization: An automated alert was configured, upgraded from scheduled to real-time mode, which fired 4 High-severity alerts during live testing. A multi-panel SOC dashboard was built to visualize failed login trends, top attacker IPs, and attack rate per minute in real time.

Result: The brute-force attack was successfully simulated, logs were captured and indexed, the detection rule fired correctly, real-time alerts triggered during live testing, and the dashboard visualized the full attack activity end-to-end, demonstrating a complete detect, alert, and visualize SIEM workflow.

2. Objectives

The primary objectives of this project were:

- Simulate a realistic SSH brute-force attack in a safe, isolated lab environment.
- Collect and ingest Linux authentication logs into Splunk Enterprise for analysis.
- Build detection logic using SPL to identify brute-force activity, starting with a count threshold and upgrading to time-based detection.
- Configure automated alerting to notify the SOC when attack thresholds are exceeded.
- Design and deploy a SOC monitoring dashboard with multiple panels to visualize the attack.
- Demonstrate understanding of the full SIEM detection workflow from log source to alert.

3. Environment Setup

The following environment was used for this project:

Operating System	Ubuntu Linux (ishtiaq-Nitro-AN515-58)
SIEM Platform	Splunk Enterprise
SSH Server	OpenSSH Server (sshd)
Log Source	/var/log/auth.log
Sourcetype	linux_secure **Note: linux_secure was assigned as a custom sourcetype label in Splunk for this lab. On Ubuntu, logs are sourced from /var/log/auth.log and monitored as a flat file input."**
Attack Target	victim@127.0.0.1 (localhost)
Index	Default (index=*)

Splunk Enterprise was installed and running locally, with the /var/log/auth.log file monitored as a data input. The sshd service was active on the machine, allowing SSH connections to be made to localhost for attack simulation.

4. Attack Simulation

To generate real brute-force data, SSH login attempts were performed against the local victim user account on 127.0.0.1. The project started with manual SSH attempts using incorrect passwords, which generated the initial failed login entries in auth.log. This was later upgraded to a scripted bash loop to generate a more realistic volume of events.

4.1 Initial SSH Login Attempts

The following terminal output shows the first SSH brute-force simulation, with repeated Permission denied responses from the SSH server:

```
ishtiaq@ishtiaq-Nitro-AN515-58:~$ ssh victim@127.0.0.1
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:VJd/NTJ/REdJXYhZ1Nokm8KwHCkcgntT/MIe7563rNU.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
victim@127.0.0.1's password:
Permission denied, please try again.
victim@127.0.0.1's password:
Permission denied, please try again.
victim@127.0.0.1's password:
victim@127.0.0.1: Permission denied (publickey,password).
ishtiaq@ishtiaq-Nitro-AN515-58:~$
```

Figure 1: Initial SSH brute-force simulation showing repeated failed login attempts to victim@127.0.0.1

4.2 Log Verification (Initial)

After simulating the attack, the auth.log file was verified using grep to confirm that failed password events were written to disk:

```
grep "Failed password" /var/log/auth.log | tail
```

```
victim@127.0.0.1's password:
Permission denied, please try again.
victim@127.0.0.1's password:
Permission denied, please try again.
victim@127.0.0.1's password:
victim@127.0.0.1: Permission denied (publickey,password).
ishtiaq@ishtiaq-Nitro-AN515-58:~$ grep "Failed password" /var/log/auth.log | tail
2026-04-11T22:11:29.287210+04:00 ishtiaq-Nitro-AN515-58 sshd[82424]: Failed password for
ctim from 127.0.0.1 port 52812 ssh2
2026-04-11T22:11:34.778638+04:00 ishtiaq-Nitro-AN515-58 sshd[82424]: Failed password for
ctim from 127.0.0.1 port 52812 ssh2
2026-04-11T22:11:39.834168+04:00 ishtiaq-Nitro-AN515-58 sshd[82424]: Failed password for
ctim from 127.0.0.1 port 52812 ssh2
ishtiaq@ishtiaq-Nitro-AN515-58:~$
```

Figure 2: Auth log verification showing the initial 3 failed password entries confirmed in /var/log/auth.log

4.3 Upgraded Attack Simulation

The initial simulation only produced 3 events, which is enough to prove the concept but not realistic. The simulation was upgraded using a bash loop that fires 20 SSH connection attempts in rapid succession, producing over 35 failed authentication events within a single minute. This is much closer to what a real brute-force tool would generate.

```
for i in {1..20}; do ssh victim@127.0.0.1; done
```

```
ishtiaq@ishtiaq-Nitro-AN515-58:/opt/splunk/ishtiaq@ishtiaq-Nitro-AN515-58:/opt/splunk/bin$ grep "Failed password" /var/log/auth.log | tail -20
2026-04-20T11:53:05.772461+04:00 ishtiaq-Nitro-AN515-58 sshd[6255]: Failed password for victim from 127.0.0.1 port 41998 ssh2
2026-04-20T11:53:05.948243+04:00 ishtiaq-Nitro-AN515-58 sshd[6255]: Failed password for victim from 127.0.0.1 port 41998 ssh2
2026-04-20T11:53:06.315718+04:00 ishtiaq-Nitro-AN515-58 sshd[6258]: Failed password for victim from 127.0.0.1 port 42000 ssh2
2026-04-20T11:53:06.508517+04:00 ishtiaq-Nitro-AN515-58 sshd[6258]: Failed password for victim from 127.0.0.1 port 42000 ssh2
2026-04-20T11:53:06.890221+04:00 ishtiaq-Nitro-AN515-58 sshd[6261]: Failed password for victim from 127.0.0.1 port 42006 ssh2
2026-04-20T11:53:07.078706+04:00 ishtiaq-Nitro-AN515-58 sshd[6261]: Failed password for victim from 127.0.0.1 port 42006 ssh2
2026-04-20T11:53:07.444723+04:00 ishtiaq-Nitro-AN515-58 sshd[6264]: Failed password for victim from 127.0.0.1 port 42012 ssh2
2026-04-20T11:53:07.639613+04:00 ishtiaq-Nitro-AN515-58 sshd[6264]: Failed password for victim from 127.0.0.1 port 42012 ssh2
2026-04-20T11:53:08.029780+04:00 ishtiaq-Nitro-AN515-58 sshd[6270]: Failed password for victim from 127.0.0.1 port 42018 ssh2
2026-04-20T11:53:08.222643+04:00 ishtiaq-Nitro-AN515-58 sshd[6270]: Failed password for victim from 127.0.0.1 port 42018 ssh2
2026-04-20T11:53:08.610572+04:00 ishtiaq-Nitro-AN515-58 sshd[6290]: Failed password for victim from 127.0.0.1 port 42030 ssh2
2026-04-20T11:53:08.801563+04:00 ishtiaq-Nitro-AN515-58 sshd[6290]: Failed password for victim from 127.0.0.1 port 42030 ssh2
2026-04-20T11:53:09.163543+04:00 ishtiaq-Nitro-AN515-58 sshd[6293]: Failed password for victim from 127.0.0.1 port 42042 ssh2
2026-04-20T11:53:09.375331+04:00 ishtiaq-Nitro-AN515-58 sshd[6293]: Failed password for victim from 127.0.0.1 port 42042 ssh2
2026-04-20T11:53:09.791551+04:00 ishtiaq-Nitro-AN515-58 sshd[6297]: Failed password for victim from 127.0.0.1 port 42050 ssh2
2026-04-20T11:53:10.070680+04:00 ishtiaq-Nitro-AN515-58 sshd[6297]: Failed password for victim from 127.0.0.1 port 42050 ssh2
2026-04-20T11:53:10.487252+04:00 ishtiaq-Nitro-AN515-58 sshd[6300]: Failed password for victim from 127.0.0.1 port 42062 ssh2
2026-04-20T11:53:10.687472+04:00 ishtiaq-Nitro-AN515-58 sshd[6300]: Failed password for victim from 127.0.0.1 port 42062 ssh2
2026-04-20T11:53:11.085256+04:00 ishtiaq-Nitro-AN515-58 sshd[6303]: Failed password for victim from 127.0.0.1 port 42076 ssh2
```

Figure 3: Upgraded attack simulation showing 20+ failed SSH login events within seconds, all logged to auth.log

5. Log Collection & Ingestion

Splunk Enterprise was configured to monitor /var/log/auth.log as a data input with the sourcetype set to linux_secure. This allowed Splunk to continuously index new authentication events as they were written to the log file.

A Splunk search was performed using the query below to confirm that the failed password events had been successfully ingested:

```
index=* "Failed password"
```

The initial search returned 3 matching events, confirming successful log ingestion from the first simulation. After the upgraded bash loop simulation, this count grew to 40 events, showing the monitoring input was picking up events in real time.

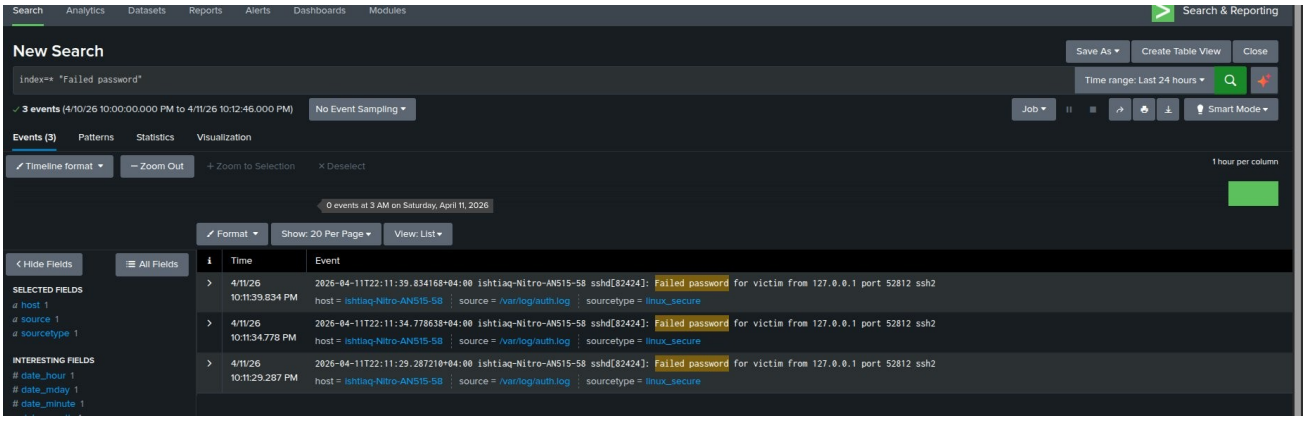


Figure 4: Splunk search results showing 3 failed password events indexed from /var/log/auth.log during initial simulation

Key fields identified by Splunk:

- host: ishtiaq-Nitro-AN515-58
- source: /var/log/auth.log
- sourcetype: linux_secure

6. Detection Logic

The detection logic was developed in two stages. The initial query used a simple count threshold to identify brute-force activity. This was later upgraded to a time-based approach which is more realistic and actionable in a real SOC environment.

6.1 Original SPL Detection Query

```
index=* "Failed password"
| rex "from (?<ip>\d+\.\d+\.\d+\.\d+)"
| stats count by user, ip
| where count > 3
```

Note: index=* was used as a lab simplification since only one index existed in this environment. In a production SOC, a specific index name would be used to improve search performance.

This query correctly identified failed password events and extracted source IP addresses, then flagged any IP with more than 3 total failures. It worked for the initial demonstration but had a limitation: it counted total failures across any time window, meaning it could not distinguish between a slow attacker spread over hours and a rapid burst attack happening in seconds.

6.2 Upgraded SPL Detection Query

```
index=* "Failed password"
| rex "Failed password for (?<user>\w+) from (?<ip>\d+\.\d+\.\d+\.\d+)"
| bin _time span=1m
| stats count by _time, ip, user
| where count > 5
```

The upgraded query adds time-based bucketing using bin _time span=1m, which groups events into 1-minute windows. It also uses a more precise regex that extracts the username field properly. The result is a query that measures the rate of attack, not just the total count, which is how real brute-force detection works in production environments.

6.3 Query Explanation

`index=* "Failed password"`	Searches all indexes for SSH authentication failure events.
`| rex "Failed password for..."`	Extracts both the username and source IP address from the raw log message using regex.
`| bin _time span=1m`	Buckets events into 1-minute windows so the query measures rate, not just total count.
`| stats count by _time, ip, user`	Counts failures per unique IP and username combination within each minute.
`| where count > 5`	Flags only results where more than 5 failures occurred within a single minute.

The upgraded query returned a single result: IP 127.0.0.1, user victim, count 35, all within the 11:53 AM minute bucket. This confirmed the detection was working correctly against a realistic attack volume.

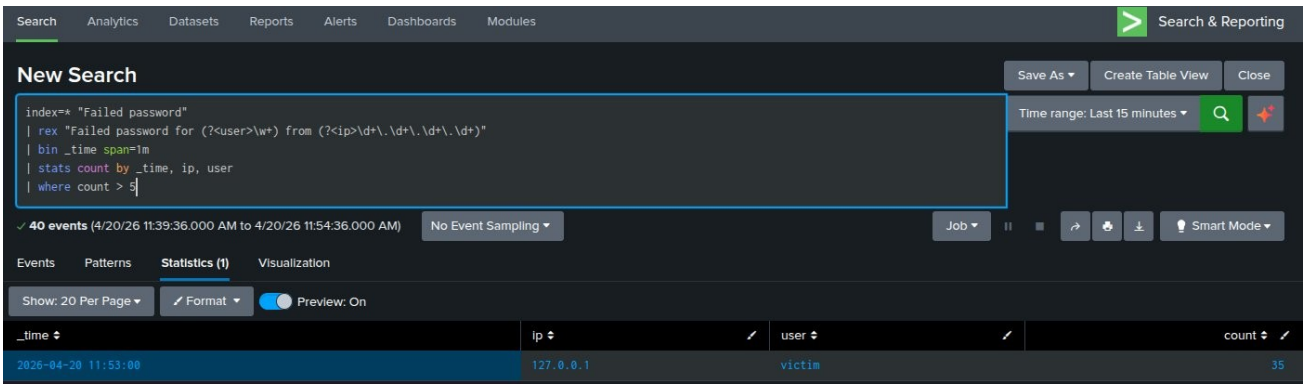


Figure 5: Upgraded SPL query results showing 35 failed login attempts from 127.0.0.1 in a single minute

7. Alert Configuration

Two alert configurations were set up across the two phases of the project. The original used a scheduled hourly alert, which was later upgraded to a real-time alert that fires immediately when the threshold is crossed.

7.1 Original Alert Configuration

The first alert was named Brute Force SSH Login Detection and configured with the following parameters:

Alert Name	Brute Force SSH Login Detection
Alert Type	Scheduled
Schedule	Run every hour, at 0 minutes past the hour
Expires	24 hours
Trigger Condition	Number of Results is greater than 3
Trigger	For each result
Throttle	Enabled, suppress for 5 minutes
Severity	High
Action	Add to Triggered Alerts

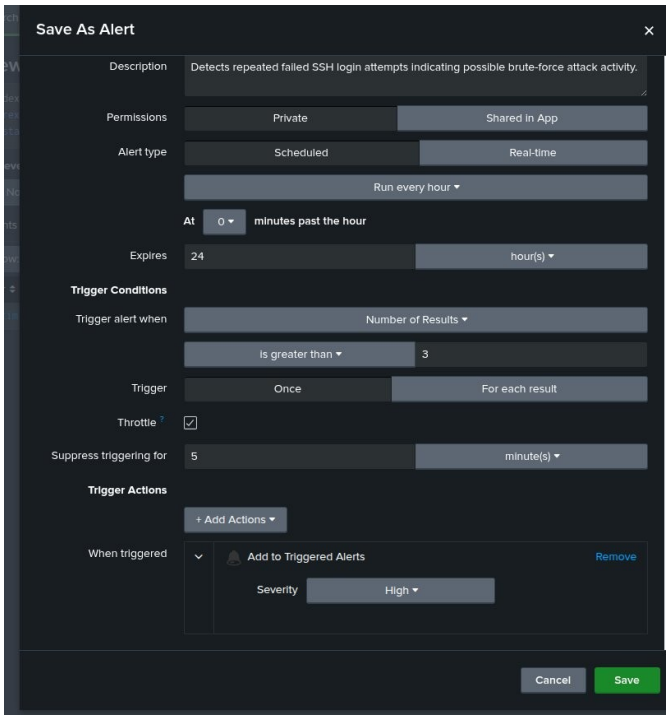


Figure 6: Original Splunk alert configuration showing trigger conditions, severity, and actions

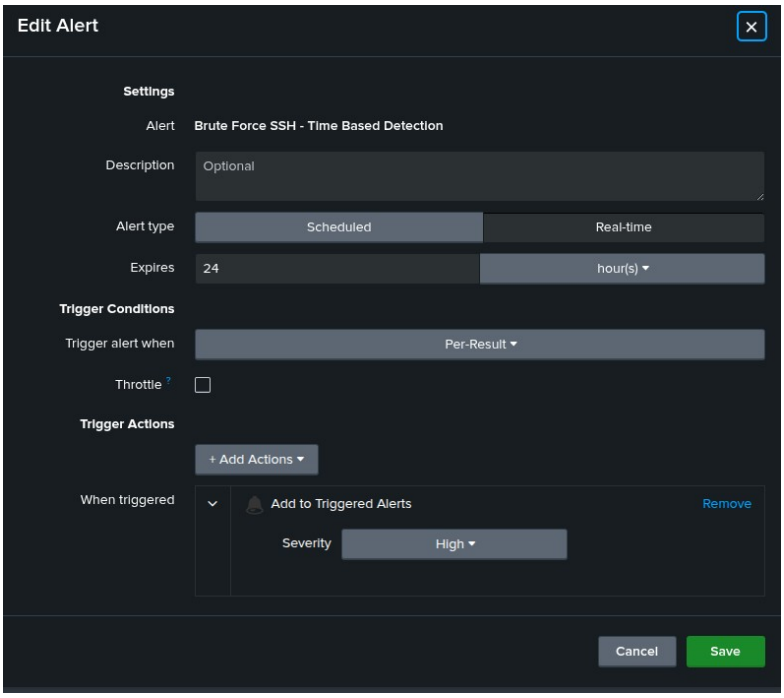
Rather than modifying the original alert, a second separate alert was created with an upgraded configuration to preserve the original for comparison.

7.2 Upgraded Alert Configuration

The alert was upgraded to real-time mode with per-result triggering, which means it fires the moment a result is returned by the detection query rather than waiting for the next scheduled run. This is significantly more useful in a real SOC environment where response time matters.

Alert Name	Brute Force SSH - Time Based Detection
Alert Type	Real-time (upgraded from Scheduled)
Trigger Condition	Per-Result
Severity	High
Action	Add to Triggered Alerts

During live testing, the real-time alert fired 4 times within seconds of each other, confirming that the detection was working as expected and that a SOC analyst would have been notified immediately during an active attack.



Note: In the screenshot the Scheduled button appears highlighted as a UI default display. The alert was configured and saved as Real-time with Per-Result triggering.

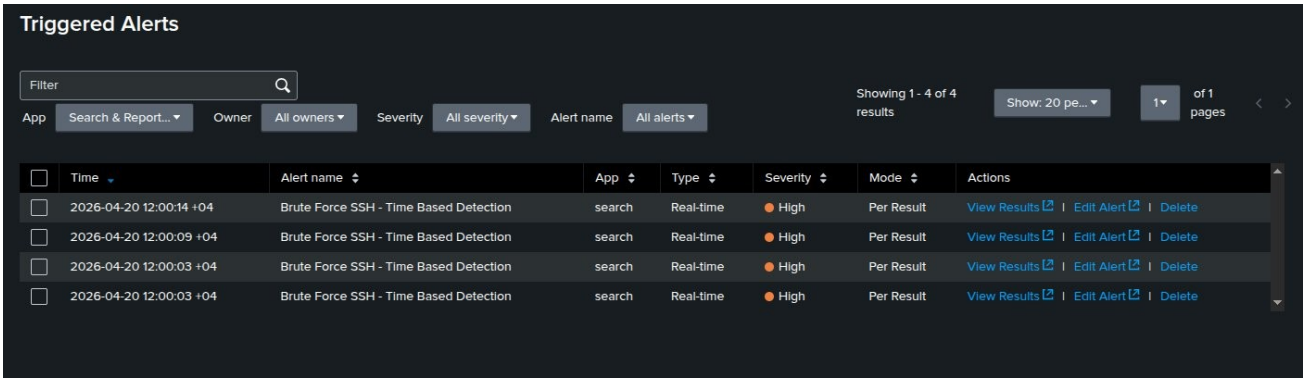


Figure 7: Triggered Alerts panel showing 4 real-time High-severity alerts fired during live attack simulation

8. SOC Dashboard

A dedicated SOC dashboard titled Brute Force SSH Monitoring was built in Splunk to provide real-time visibility into SSH attack activity. The dashboard was built initially with four panels and later expanded with a fifth panel showing the time-based attack rate.

8.1 Dashboard Overview

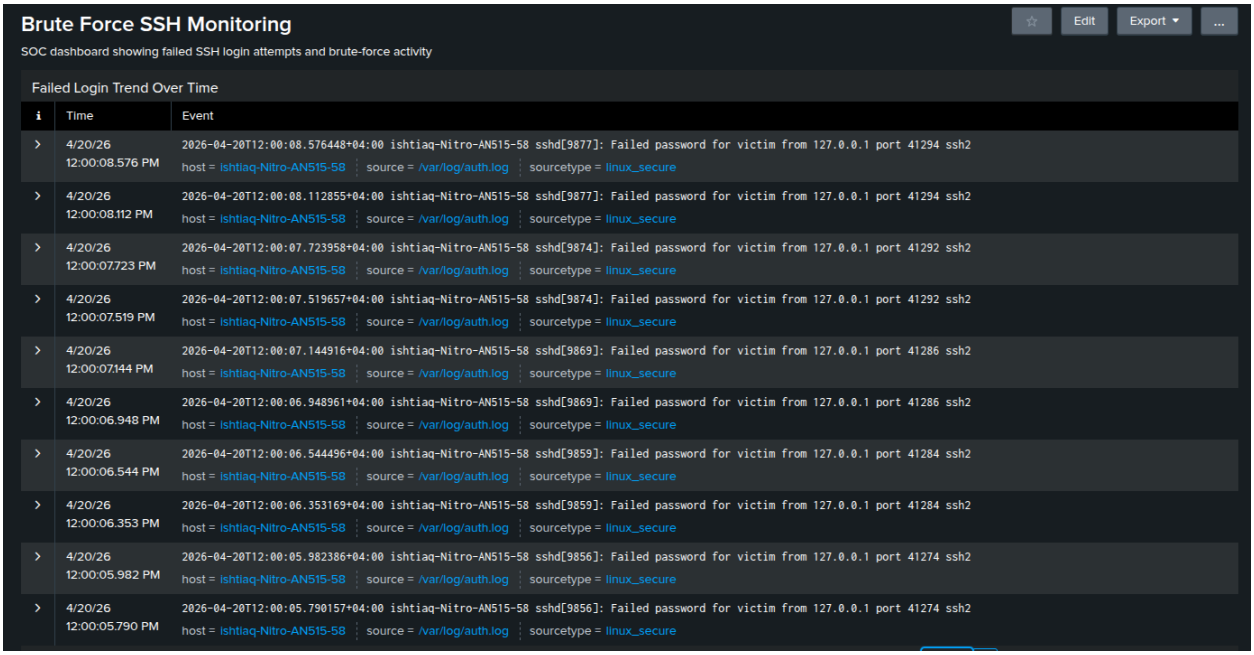


Figure 8: Full Splunk SOC dashboard showing the Brute Force SSH Monitoring dashboard with all panels

8.2 Dashboard Panels

Failed Login Trend Over Time	Displays raw failed login events in a time-stamped table, showing each individual authentication failure with full event details, host, source, and sourcetype.
Top Source IP Addresses	A **time-series line chart** tracking how the volume of attempts from each source IP changed over the 24-hour monitoring period — shows *when* the attack happened
Top Attacker IPs	A **horizontal bar chart** ranking source IPs by total failed login count — shows *who* attacked the most, useful for quick triage
Total Failed Logins	An aggregate count visualization showing the total volume of failed login events across the monitored period.
Brute Force Attempts Per Minute [Upgraded]	A line chart using the time-based SPL query, showing attempts bucketed per minute. The spike at 11:53 AM corresponds directly to the simulated attack burst.

8.3 Dashboard Panels Detail

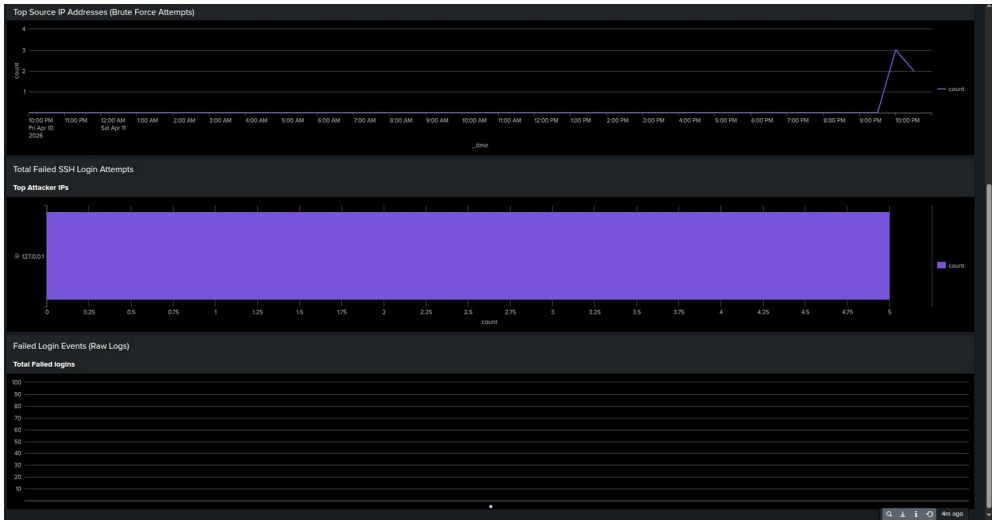


Figure 9: Dashboard panels showing Top Source IPs graph, Top Attacker IPs bar chart, and Total Failed Logins

8.4 Alert panel

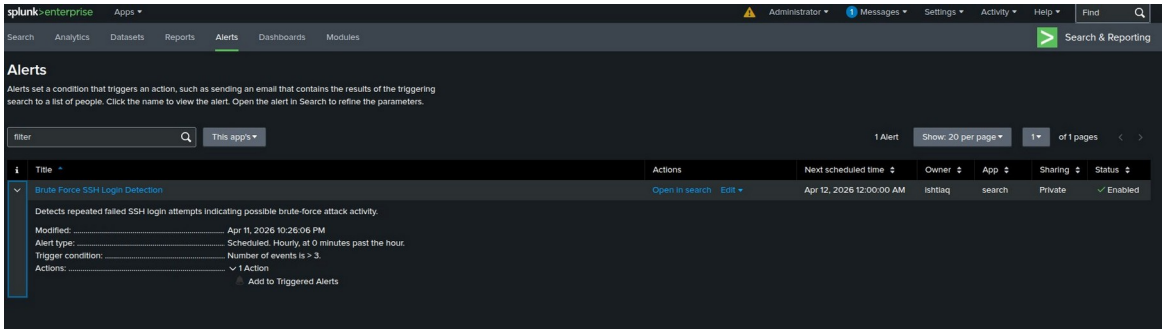


Figure 10: Alerts panel showing the Brute Force SSH Login Detection alert with trigger conditions, schedule, and enabled status

8.5 Upgraded Panel: Brute Force Attempts Per Minute

The most significant dashboard addition is the Brute Force Attempts Per Minute line chart, added during the upgrade phase. The chart clearly shows the attack spike at 11:53 AM where 35 attempts landed in a single minute, followed by a second smaller spike from a follow-up test run. This kind of visualization makes it immediately obvious when an attack is happening without needing to read through raw logs.

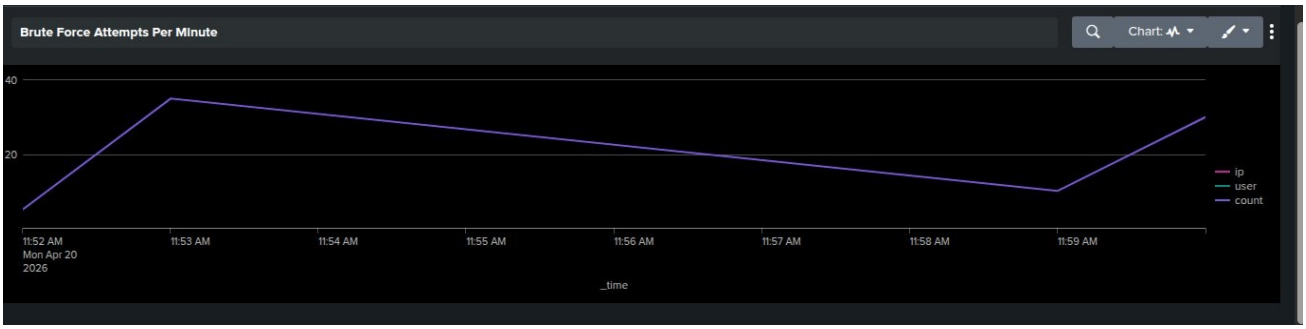


Figure 11: New dashboard panel showing brute-force attempts per minute with a clear attack spike at 11:53 AM

9. Challenges & Fixes

The following challenges were encountered during the project and resolved as described:

Challenge	Fix Applied
SSH service not running initially	Installed and started OpenSSH server using apt install openssh-server and systemctl start ssh.
Splunk disk space warnings	Cleared unnecessary files and temp data to free up disk space for log indexing.
Indexing time range issues	Adjusted the Splunk search time range to Last 24 hours to ensure newly indexed events were captured.
Host key verification prompt	Accepted the SSH host key fingerprint on first connection to allow the simulation to proceed.
Low event count from manual simulation (only 3 events)	Replaced manual SSH attempts with a bash loop generating 20 rapid connection attempts, producing 35+ events in a single minute.
Scheduled alert not reactive enough	Switched alert type from Scheduled hourly to Real-time with Per-Result trigger so alerts fire immediately when the threshold is crossed.

10. Results

The project achieved all of its intended objectives. The table below summarises the outcomes:

Objective	Result	Notes
Brute-force attack simulated	**SUCCESS**	Upgraded from manual attempts to bash loop generating 35+ events per minute.
Logs captured in auth.log	**SUCCESS**	Verified with grep, 40 events confirmed after upgrade.
Splunk ingestion confirmed	**SUCCESS**	40 events indexed, visible in Splunk with correct sourcetype.
Detection query functional	**SUCCESS**	Upgraded SPL uses time-based bucketing and proper username extraction.
Alert triggered on threshold	**SUCCESS**	Real-time alert fired 4 times during live testing.
Dashboard visualized attack	**SUCCESS**	5-panel dashboard includes per-minute attack rate chart.

The detect, alert, and visualize SIEM workflow was fully demonstrated in a controlled lab environment, confirming that Splunk Enterprise is capable of identifying and surfacing SSH brute-force activity in real time.

11. What I Learned

This project taught me things that are difficult to pick up just from reading documentation or watching tutorials. Actually building and running each component made the concepts stick in a way that theory alone does not.

- How a SIEM transforms raw log lines into actionable intelligence. Watching a failed SSH attempt in a terminal become a Splunk alert made the entire workflow make sense in a practical way.
- Writing SPL from scratch is a skill on its own. Building the regex extraction, time bucketing, and threshold logic step by step gave me a real understanding of how detection rules are constructed, not just what they look like when someone else writes them.
- Time range configuration in Splunk is easy to get wrong. Early on I was getting zero results not because the query was wrong, but because the search window was set incorrectly. Small configuration details matter.
- The difference between a scheduled alert and a real-time alert is significant in practice. An hourly scheduled alert in a live attack scenario could give an attacker nearly 60 minutes of undetected access. Real-time is not just a nice feature, it is necessary for genuine incident response.
- Low event volume is a real limitation in lab environments. The upgrade from 3 events to 35+ per minute changed what the dashboard could actually show and made the detection logic feel meaningful rather than just a proof of concept.

12. What I Would Do Better

This project is still evolving. The following are honest gaps I identified and areas I plan to build on:

- The attack is still localhost to localhost. A proper two-VM setup with a dedicated attacker machine and a separate victim would make the simulation significantly more realistic, with a genuine external IP appearing in the logs.
- I plan to use Hydra or a similar tool to automate the attack at scale, generating hundreds of attempts with different usernames and passwords, which is much closer to what real brute-force tools actually do.
- Adding geolocation lookup using the iplocation command in SPL would allow the dashboard to show attacker locations on a map panel, which is a common feature in real SOC dashboards.
- A detection panel for successful logins that follow a series of failures would be a valuable addition. That pattern is the most dangerous because it means the brute-force actually worked.
- The next step after detection and alerting is automated response. A script that triggers ufw to block the attacking IP when the alert fires would turn this from a detect and alert workflow into detect, alert, and respond, which moves into SOAR territory.

Each upgrade has taught me something new. The process of building something, finding its weaknesses, and going back to fix them is exactly the mindset I want to keep developing.

13. Conclusion

This project provided genuine hands-on experience with the core components of a Security Operations Center workflow using Splunk Enterprise as the SIEM platform. By simulating a real SSH brute-force attack, ingesting authentication logs, writing custom SPL detection logic, configuring real-time alerts, and building a monitoring dashboard, the full lifecycle of threat detection was demonstrated end-to-end.

The key takeaway is how a SIEM platform centralizes log data and transforms raw events into actionable intelligence, turning individual failed login lines in auth.log into a prioritized High-severity alert visible to a SOC analyst. The upgrade from a basic count-based detection to a time-based approach reinforced the importance of understanding not just how to write a query, but why one approach is more effective than another in a real environment.

In a real-world SOC environment, this detection rule would be one layer in a broader defence-in-depth strategy, complemented by IP blocking, rate limiting, and multi-factor authentication to reduce the attack surface further. That is exactly the direction this project will continue to grow.